Knowledge and Data

Working Group

Week 5: Knowledge Engineering

Question 1—Ontology

Task: Explain the concept of an ontology.

Question 1—Ontology

Task: Explain the concept of an ontology.

Ontology:

An explicit specification of a shared conceptualisation that holds in a particular context.

Characteristics?

Question 1—Ontology

Task: Explain the concept of an ontology.

Ontology:

An explicit specification of a shared conceptualisation that holds in a particular context.

Characteristics:

explicit	- a formal, machine interpretable language
shared	- consensus on the definitions, shared openly on the web
conceptualisation	- defined classes, instances and relations
context	- tied to a particular domain or task

NB: schema and TBox are terms that are also often associated with an ontology.

NB2: because ontologies can be represented using RDF, the distinction between ontologies and instances in knowledge graphs can be less clear.

Question 2—Ontology Engineering

Consider the domain of a scheduling study programmes

Task: Describe the standard steps of the ontology engineering process and give two examples per step (except anomalies)

Question 2—Ontology Engineering

Consider the domain of a scheduling study programmes

Task: Describe the standard steps of the ontology engineering process and give two examples per step (except anomalies)

1 Determine domain & scope
2 Consider reuse
3 Enumerate terms
4 Define taxonomy
5 Define properties
6 Define classes and properties
7 Define instances
8 Check for anomalies

Question 2—Ontology Engineering

Domain: scheduling study programmes

1) Determine domain & scope

Something along this line:

"A study programme runs for one or more years, and consists of one or more courses. These courses are taught by teachers in a certain room, start and end at a specific time, and are provided during one or more periods throughout the academic year. Only study programmes from universities are considered, encompassing Bachelor and Master level. ..."

Question 2—Ontology Engineering

Domain: scheduling study programmes

2) Consider reuse

Something along this line:

"By modelling the ontology in a generic way it can be reused by most universities that offer study programmes, both nationally and abroad."

Question 2—Ontology Engineering

Domain: scheduling study programmes

3) Enumerate terms

- Courses,
- Teachers,
- Rooms
- taught by / teaches
- starts at / stops at
- etc

Question 2—Ontology Engineering

Domain: scheduling study programmes

4) Define taxonomy

- Course
  - BSc Course
    - 1st Year
    - 2nd Year
    - 3rd Year
      - 1st Period
      - 2nd Period
      - 3rd Period
      - ...
  - MSc Course
    - ...

Question 2—Ontology Engineering

Domain: scheduling study programmes

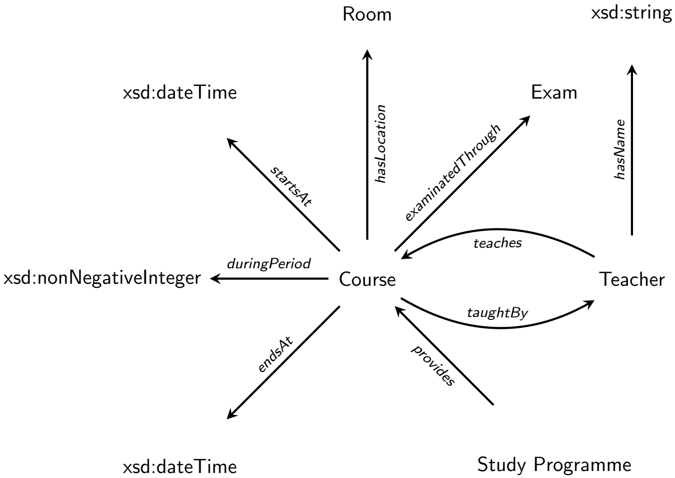
5) Define properties

- taughtBy / teaches
- startsAt / endsAt
- duringPeriod
- hasLocation
- examinatedThrough
- etc

Question 2—Ontology Engineering

Domain: scheduling study programmes

6) Define classes and their properties



Question 2—Ontology Engineering

Domain: scheduling study programmes

7) Define instances

Programmes:	Artificial Intelligence, Information Sciences, Business Analytics, . . .
Courses:	Knowledge&Data, Intelligent Systems, Computational Thinking, . . .
Teachers:	Victor, Jieying, Benno, Xander, . . .
Rooms:	WN-KC159, NU-10A89, HG-09A01, . . .
etc	

Question 3—Ontological Commitment

Explain, in your own words, the concept of over-commitment and why it is ill-advised.

Question 3—Ontological Commitment

Explain, in your own words, the concept of over-commitment and why it is ill-advised.

Over-committed axioms make too strong a statements of the domain they model. In an open world, this can have unforeseen consequences.

Example?

Question 3—Ontological Commitment

Explain, in your own words, the concept of over-commitment and why it is ill-advised.

Over-committed axioms make too strong a statements of the domain they model. In an open world, this can have unforeseen consequences.

Example:

- Cars must have exactly four wheels
- Cats must have a hair colour
- Planets must orbit a star
- ...

Rule of thumb: choose the minimal ontological commitment needed

Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	
class	class	
individual	class	
property	property	
property	individual / class	

Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	owl:sameAs owl:differentFrom owl:FunctionalProperty owl:InverseFunctionalProperty
class	class	
individual	class	
property	property	
property	individual / class	

Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	owl:sameAs
		owl:differentFrom
		owl:FunctionalProperty
		owl:InverseFunctionalProperty
class	class	owl:equivalentClass
		rdfs:subClassOf
		owl:disjointWith
individual	class	
property	property	
property	individual / class	

Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	owl:sameAs owl:differentFrom owl:FunctionalProperty owl:InverseFunctionalProperty
class	class	owl:equivalentClass rdfs:subClassOf owl:disjointWith
individual	class	rdf:type class restriction punning
property	property	
property	individual / class	

Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	owl:sameAs
		owl:differentFrom
		owl:FunctionalProperty
		owl:InverseFunctionalProperty
class	class	owl:equivalentClass
		rdfs:subClassOf
		owl:disjointWith
individual	class	rdf:type
		class restriction
		punning
property	property	rdfs:subPropertyOf
		owl:equivalentProperty
		owl:disjointProperty
property	individual / class	

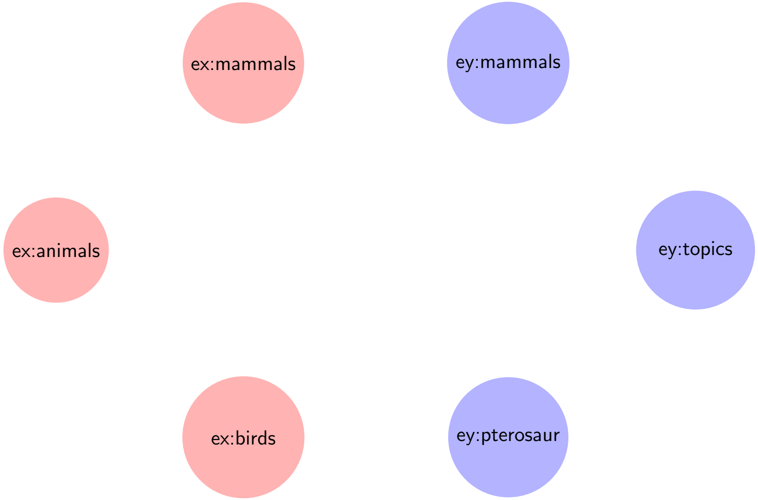
Question 4—Mapping ontologies

Task: Name the possible mapping pairs and the properties that can be used to align them.

source	*target*	*properties*
individual	individual	owl:sameAs
		owl:differentFrom
		owl:FunctionalProperty
		owl:InverseFunctionalProperty
class	class	owl:equivalentClass
		rdfs:subClassOf
		owl:disjointWith
individual	class	rdf:type
		class restriction
		punning
property	property	rdfs:subPropertyOf
		owl:equivalentProperty
		owl:disjointProperty
property	individual / class	owl:propertyChainAxiom
		punning

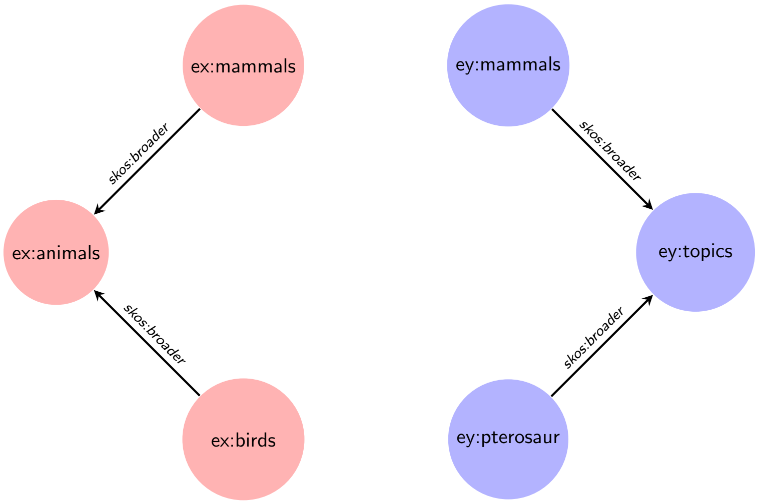
Question 5—Mapping ontologies

Task: Add the SKOS mapping relations



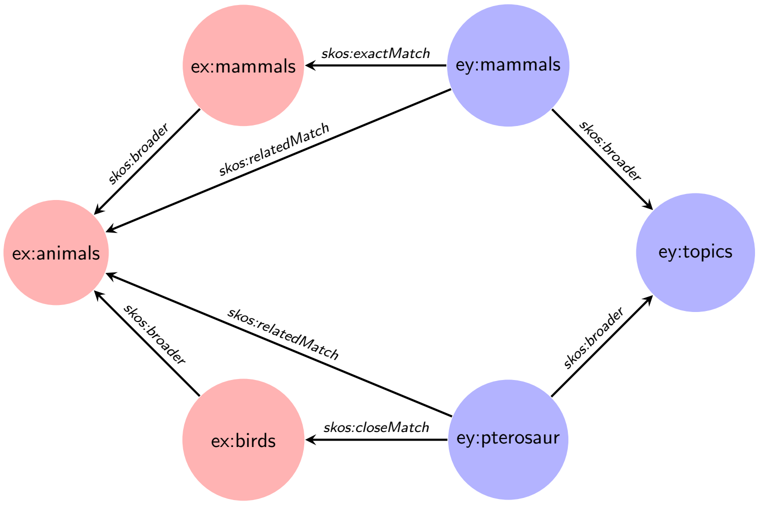
Question 5—Mapping ontologies

Task: Add the SKOS mapping relations



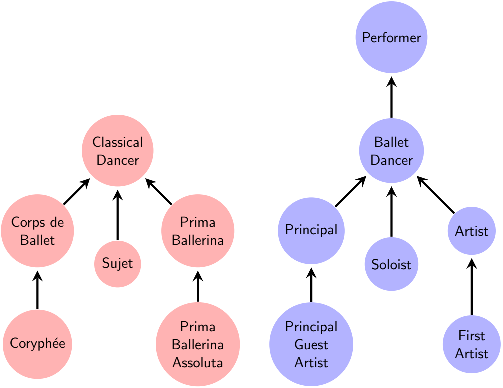
Question 5—Mapping ontologies

Task: Add the SKOS mapping relations



Question 5a—Mapping ontologies

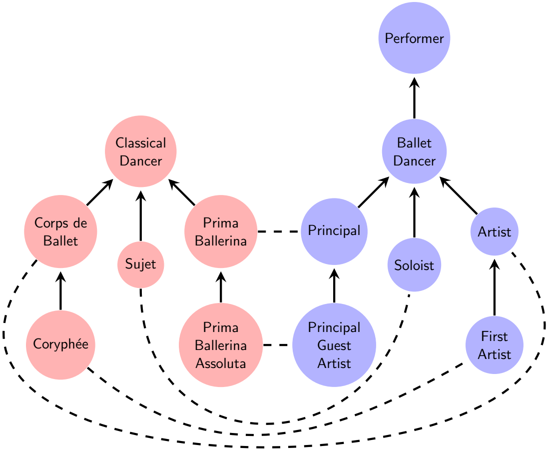
Task: what is the correct mapping in this scenario



1) owl:equivalentClass 2) owl:sameAs 3) owl:disjointWith

Question 5a—Mapping ontologies

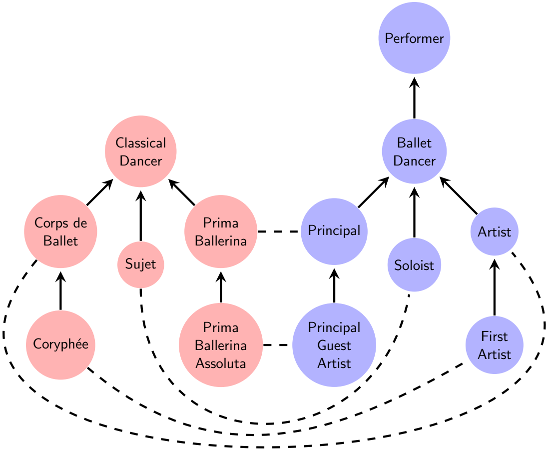
Task: what is the correct mapping in this scenario



1) owl:equivalentClass 2) owl:sameAs 3) owl:disjointWith

Question 5a—Mapping ontologies

Task: what is the correct mapping in this scenario

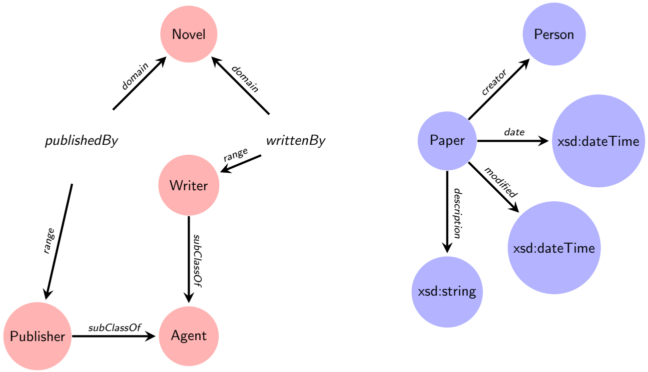


1) owl:equivalentClass 2) owl:sameAs 3) owl:disjointWith

`owl:equivalentClass`, as these the classes have the same members

Question 5b—Mapping ontologies

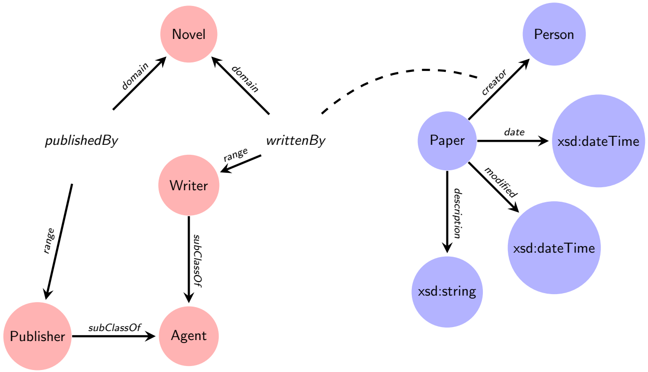
Task: what is the correct mapping in this scenario



1) skos:closeMatch 2) rdfs:subPropertyOf 3) owl:equivalentProperty

Question 5b—Mapping ontologies

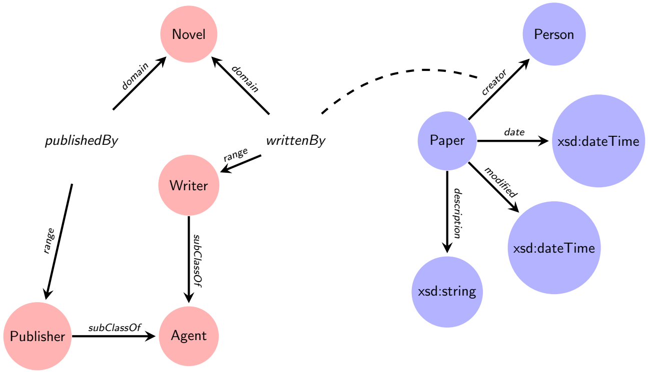
Task: what is the correct mapping in this scenario



1) skos:closeMatch 2) rdfs:subPropertyOf 3) owl:equivalentProperty

Question 5b—Mapping ontologies

Task: what is the correct mapping in this scenario

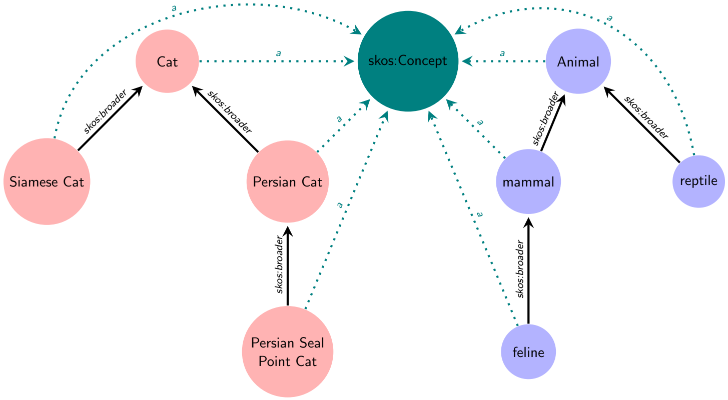


1) skos:closeMatch 2) rdfs:subPropertyOf 3) owl:equivalentProperty

`rdfs:subPropertyOf`, as these a) are properties and b) one is more specific than the other

Question 5c—Mapping ontologies

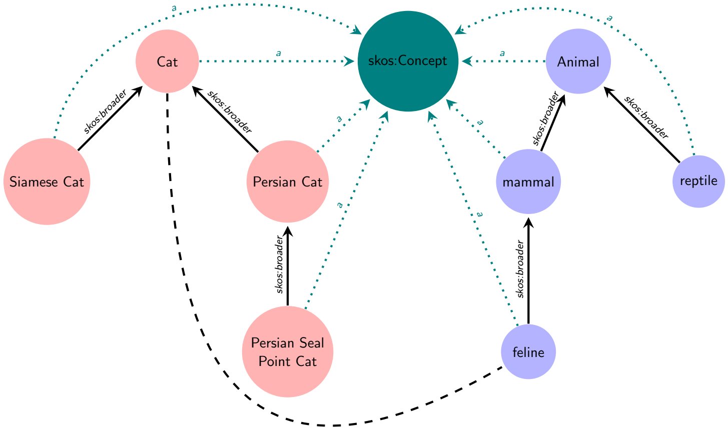
Task: what is the correct mapping in this scenario



1) owl:sameAs 2) rdfs:subClassOf 3) skos:broadMatch

Question 5c—Mapping ontologies

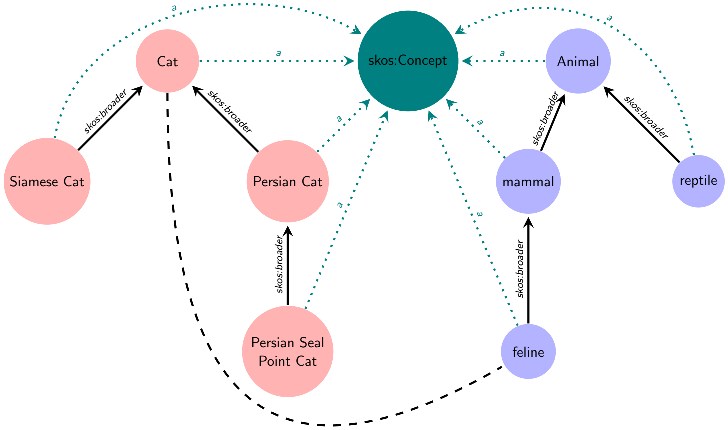
Task: what is the correct mapping in this scenario



1) owl:sameAs 2) rdfs:subClassOf 3) skos:broadMatch

Question 5c—Mapping ontologies

Task: what is the correct mapping in this scenario



1) owl:sameAs 2) rdfs:subClassOf 3) skos:broadMatch

`skos:broadMatch`, as a) these are SKOS concepts, b) one is a broader concept than the other

Question 6—Data integration with reasoning

Consider the following two graphs, G and H:

G:			H:		
(ont1:v,	ont1:s,	ont1:a) .	(ont2:b,	ont2:r,	ont2:o) .
(ont1:a,	ont1:p,	"value X") .	(ont2:b,	ont2:q,	"value X") .

An expert tells you that `ont1:p` is the same property as `ont2:q`.

Task: Name which axioms you need to add to integrate G and H, such that SPARQL query

```
SELECT ?x
WHERE {
  ont1:v ont1:s ?m .
  ?m     ont2:r ?x .
}
```

returns `ont2:o`. NB: use only axioms defined on properties

Question 6—Data integration with reasoning

Consider the following two graphs, G and H:

G:

(ont1:v, ont1:s, ont1:a) .
(ont1:a, ont1:p, "value X") .

H:

(ont2:b, ont2:r, ont2:o) .
(ont2:b, ont2:q, "value X") .

An expert tells you that `ont1:p` is the same property as `ont2:q`.

Task: Name which axioms you need to add to integrate G and H, such that SPARQL query

```
SELECT ?x
WHERE {
  ont1:v ont1:s ?m .
  ?m     ont2:r ?x .
}
```

returns `ont2:o`.

Answer (partial):

(ont1:p, owl:equivalentProperty, ont2:q).

NB: use only axioms defined on properties

Question 6—Data integration with reasoning

Consider the following two graphs, G and H:

G:

(ont1:v, ont1:s, ont1:a) .
(ont1:a, ont1:p, "value X") .

H:

(ont2:b, ont2:r, ont2:o) .
(ont2:b, ont2:q, "value X") .

An expert tells you that `ont1:p` is the same property as `ont2:q`.

Task: Name which axioms you need to add to integrate G and H, such that SPARQL query

```
SELECT ?x
WHERE {
  ont1:v ont1:s ?m .
  ?m     ont2:r ?x .
}
```

returns `ont2:o`.

Answer:

(ont1:p, owl:equivalentProperty, ont2:q).
and
(ont2:q, rdf:type, owl:inverseFunctionalProperty).

NB: use only axioms defined on properties

Question 6—Data integration with reasoning

Consider the following two graphs, G and H:

G:			H:		
(ont1:v,	ont1:s,	ont1:a) .	(ont2:b,	ont2:r,	ont2:o) .
(ont1:a,	ont1:p,	"value X") .	(ont2:b,	ont2:q,	"value X") .

An expert tells you that `ont1:p` is the same property as `ont2:q`.

Task: Name which axioms you need to add to integrate G and H, such that SPARQL query

```
SELECT ?x
WHERE {
  ont1:v ont1:s ?m .
  ?m     ont2:r ?x .
}
```

returns `ont2:o`.

Answer:

(ont1:p, owl:equivalentProperty, ont2:q).
and
(ont2:q, rdf:type, owl:inverseFunctionalProperty).

`?m` is matched against `ont1:a`, which is now inferred as being equal to `ont2:b`, such that `?x` becomes `ont2:o`.

NB: use only axioms defined on properties

Question 7—Taxonomies & Ontologies

Consider the following concepts:

- ex:Country
- ex:Asia
- ex:Capital
- ex:PopulatedPlace
- ex:TheNetherlands
- ex:Continent
- ex:Paris
- ex:City

Task: Create an ontology with these concepts, using **only** the following three properties:

- rdf:type
- rdfs:subClassOf
- skos:broader

hint: first ordering the concepts from general to specific might help.

Question 7—Taxonomies & Ontologies

Classes (ordered):

1. ex:PopulatedPlace
2. ex:Continent
3. ex:Country
4. ex:City
5. ex:Capital

Individuals:

- ex:Asia
- ex:TheNetherlands
- ex:Paris

Question 7—Taxonomies & Ontologies

Classes (ordered):

1. ex:PopulatedPlace
2. ex:Continent
3. ex:Country
4. ex:City
5. ex:Capital

Individuals:

- ex:Asia
- ex:TheNetherlands
- ex:Paris

Answer (partial):

ex:Asia,	rdf:type,	ex:Continent .
ex:TheNetherlands,	rdf:type,	ex:Country .
ex:Paris,	rdf:type,	ex:Capital .

Question 7—Taxonomies & Ontologies

Classes (ordered):

1. ex:PopulatedPlace
2. ex:Continent
3. ex:Country
4. ex:City
5. ex:Capital

Individuals:

- ex:Asia
- ex:TheNetherlands
- ex:Paris

Answer (partial):

ex:Continent,	rdfs:subClassOf,	ex:PopulatedPlace .
ex:Country,	rdfs:subClassOf,	ex:PopulatedPlace ;
ex:City,	rdfs:subClassOf,	ex:PopulatedPlace ;
ex:Capital,	rdfs:subClassOf,	ex:City .
ex:Asia,	rdf:type,	ex:Continent .
ex:TheNetherlands,	rdf:type,	ex:Country .
ex:Paris,	rdf:type,	ex:Capital .

Question 7—Taxonomies & Ontologies

Classes (ordered):

1. ex:PopulatedPlace
2. ex:Continent
3. ex:Country
4. ex:City
5. ex:Capital

Individuals:

- ex:Asia
- ex:TheNetherlands
- ex:Paris

Answer:

ex:Continent,	rdfs:subClassOf,	ex:PopulatedPlace .
ex:Country,	rdfs:subClassOf,	ex:PopulatedPlace ;
	skos:broader,	ex:Continent .
ex:City,	rdfs:subClassOf,	ex:PopulatedPlace ;
	skos:broader,	ex:Continent ;
	skos:broader,	ex:Country .
ex:Capital,	rdfs:subClassOf,	ex:City .
ex:Asia,	rdf:type,	ex:Continent .
ex:TheNetherlands,	rdf:type,	ex:Country .
ex:Paris,	rdf:type,	ex:Capital .

Quiz

To test your proficiency:

Exercise quiz Module 5

(see the course's Canvas page)

NB: completing the quiz is mandatory to continue to next week's module